R Scribbles

Anurag (bmat2508@isibang.ac.in)

```r
#using the for loop
v=c(1,2,3,4,5)
for(x in v){
    print(x)
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

```r
#For taking modulo something
5%%2
```

```
## [1] 1
```

```r
#For floor division
35%/%2
```

```
## [1] 17
```

```r
#Vectorized relational operators
v>=2
```

```
## [1] FALSE  TRUE  TRUE  TRUE  TRUE
```

```r
#We have two types of logical operators in R; the regular ones and short-circuit counterparts
x=c(FALSE,TRUE,TRUE)
y=c(TRUE,FALSE,FALSE)
#Regular and, computes logical and element-wise and returns a vector of boolean values
x&y
```

```
## [1] FALSE FALSE FALSE
```

```r
#Short-circuit and, uses the symbol && and same for or, regular one uses |
#To extract certain values from a vector
x=1:10
x[x>5]
```

```
## [1]  6  7  8  9 10
```

```r
x[c(1,2,4)]
```

```
## [1] 1 2 4
```

```r
x[3]
```

```
## [1] 3
```

```r
x[2:5]
```

```
## [1] 2 3 4 5
```

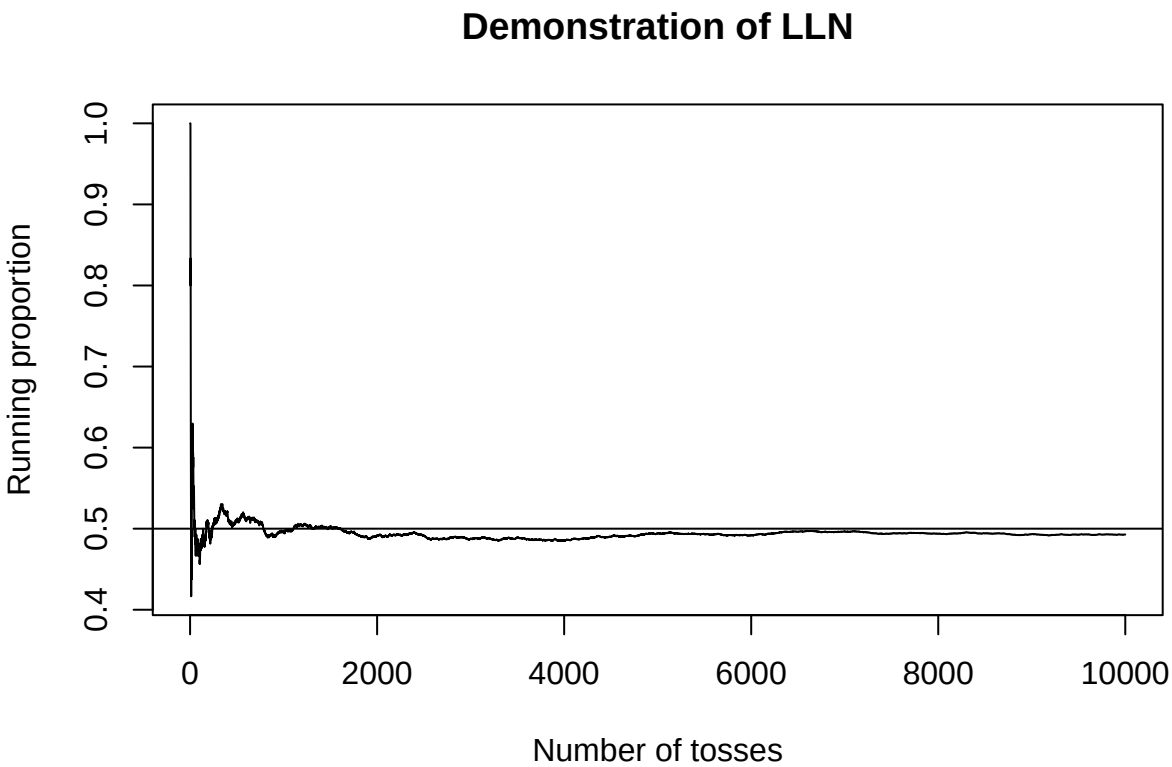
```r
#Using if-else if-else statements
x=12
if(x<0){
    print("Negative")
} else if(x==0){
    print("This is zero")
} else{
    print("Positive")
}
```

```
## [1] "Positive"
```

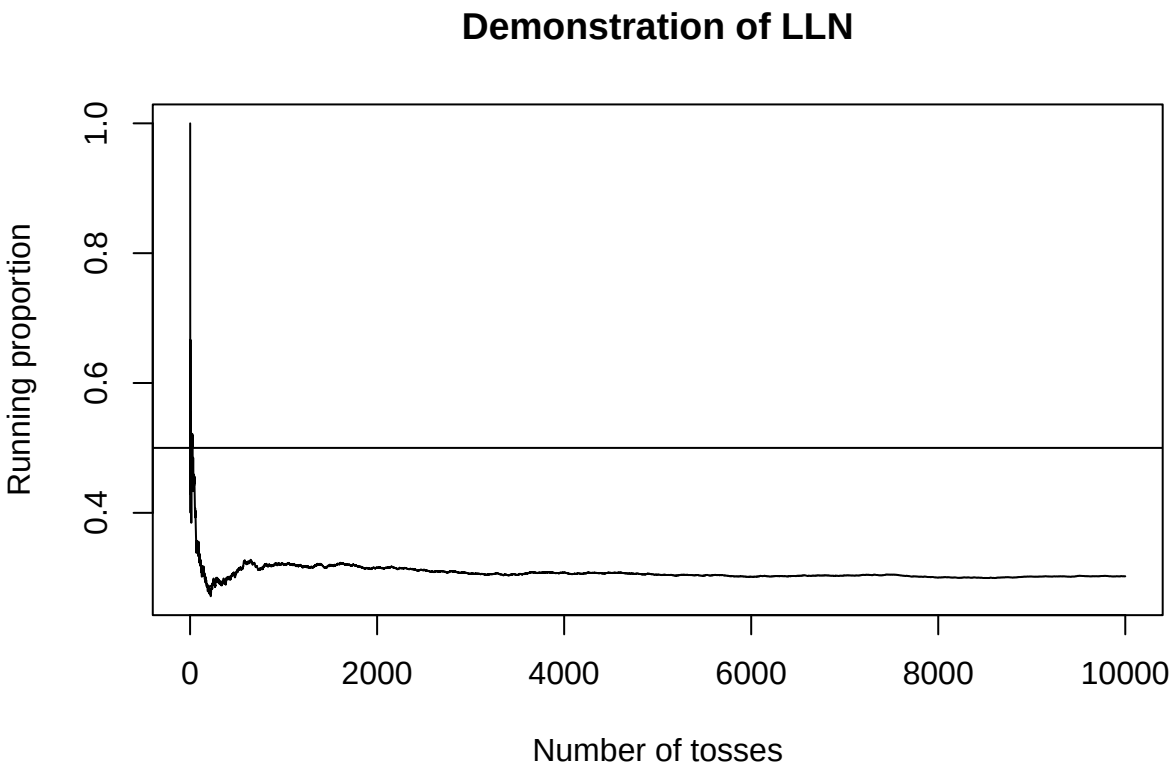
```r
x=-5
while(x<=5){
    if(x==2){
        break
    } else if(x==1){
        x=x+1
        next
    } else{
        print(x)
    }
    x=x+1
}
```

```
## [1] -5
## [1] -4
## [1] -3
## [1] -2
## [1] -1
## [1] 0
```

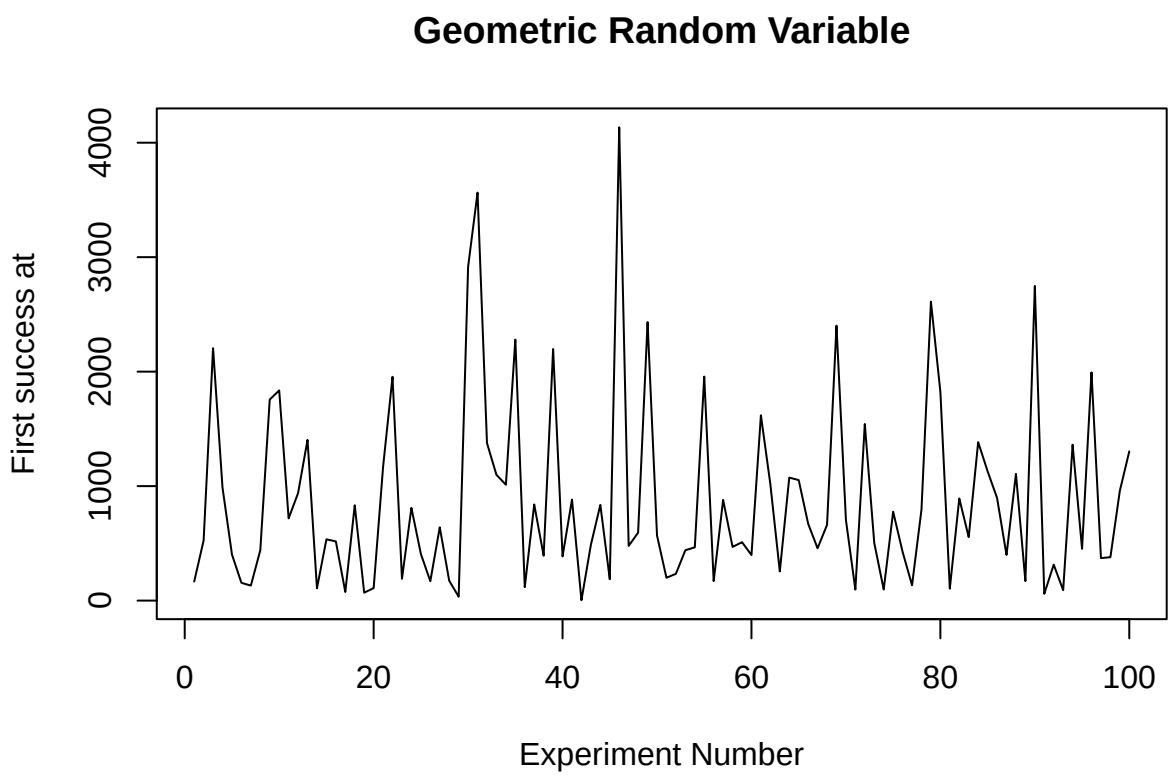
```r
#problems
#simulating law of large numbers
x=sample(c(0,1),size=10000,replace=TRUE)
c=0
h=0
p=numeric(10000)
for(i in x){
    c=c+1
    if(i==1){
        h=h+1
    }
    p[c]=h/c
}
plot(p,type="l",xlab="Number of tosses",ylab="Running proportion",main="Demonstration of LLN")
abline(h=0.5)
```



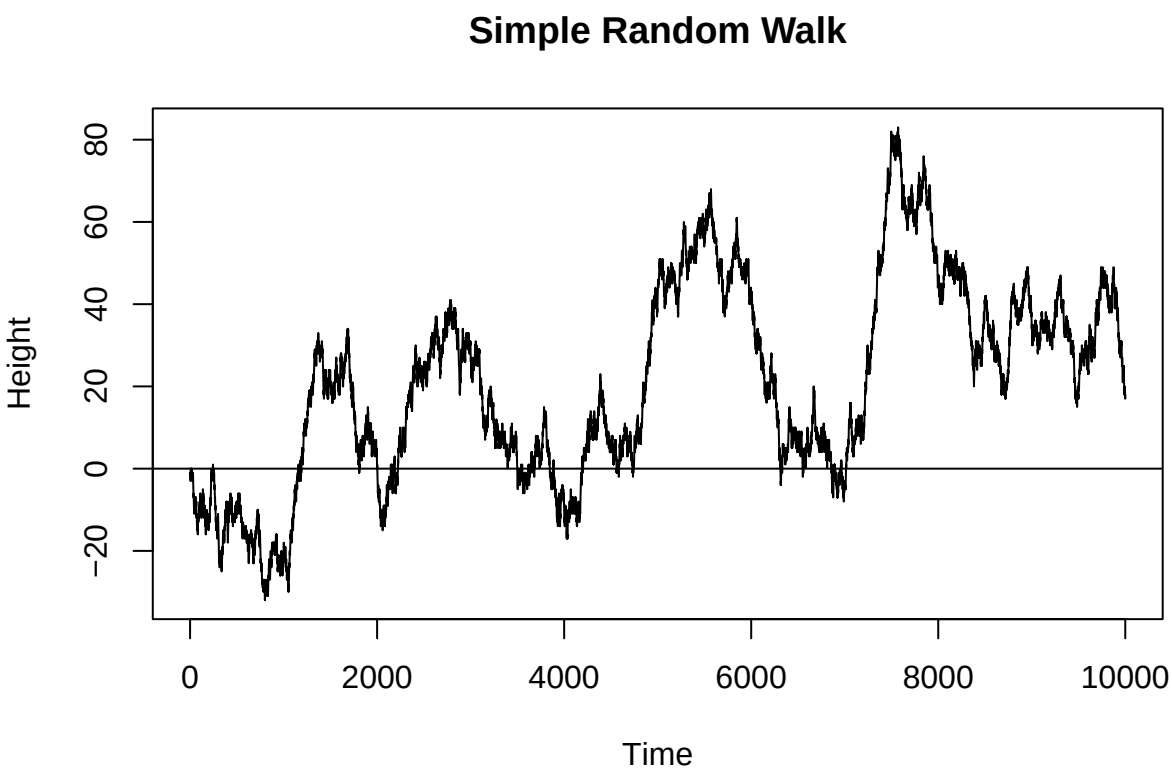

```
#One could also write abline(v=k) to draw the vertical line x=k and
#to draw a line with given intercept and slope one would write abline(a=intercept,b=slope)
#Simulating the same thing but now with a biased coin instead
x=sample(c(0,1),size=10000,replace=TRUE,prob=c(0.7,0.3))
c=0
h=0
p=numeric(10000)
for(i in x){
    c=c+1
    if(i==1){
        h=h+1
    }
    p[c]=h/c
}
plot(p,type="l",xlab="Number of tosses",ylab="Running proportion",main="Demonstration of LLN")
abline(h=0.5)
```



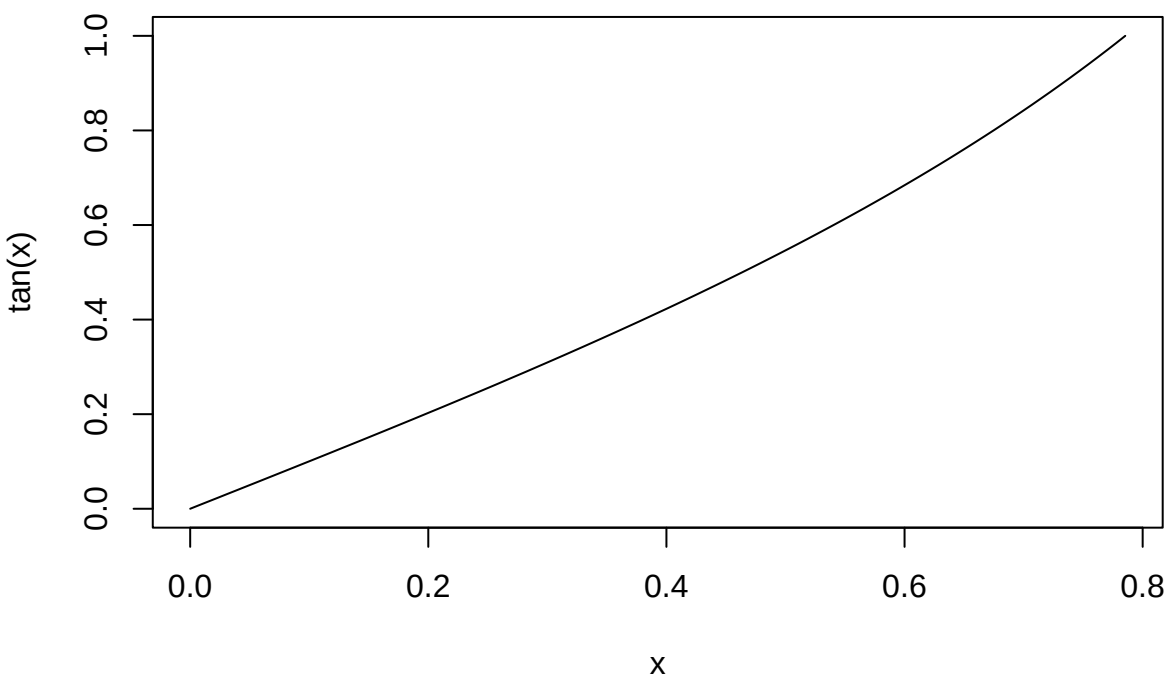

```
f=0
l=numeric(100)
while(f<100){
    c=0
    while(TRUE){
        x=sample(c(0,1),1,replace=TRUE,prob=c(0.999,0.001))
        if(x==1){
            c=c+1
            break
        } else{
            c=c+1
        }
    }
    f=f+1
    l[f]=c
}
plot(l,type="l",xlab="Experiment Number",ylab="First success at",main="Geometric Random Variable")
```




```
#First return to 0 for a SRW
n=10000
f=0
l=numeric(n)
while(f<n){
    x=sample(c(-1,1),1,replace=TRUE)
    f=f+1
    l[f]=x
}
p=cumsum(l)
plot(p,type="l",xlab="Time",ylab="Height",main="Simple Random Walk")
abline(h=0)
```




```r
#Drawing a certain function on a given interval
curve(tan,from=0,to=pi/4)
```




```r
#Optimizing a function on an interval, by default it returns the minimum
optimize(sin,c(0,pi/2),maximum=TRUE)
```

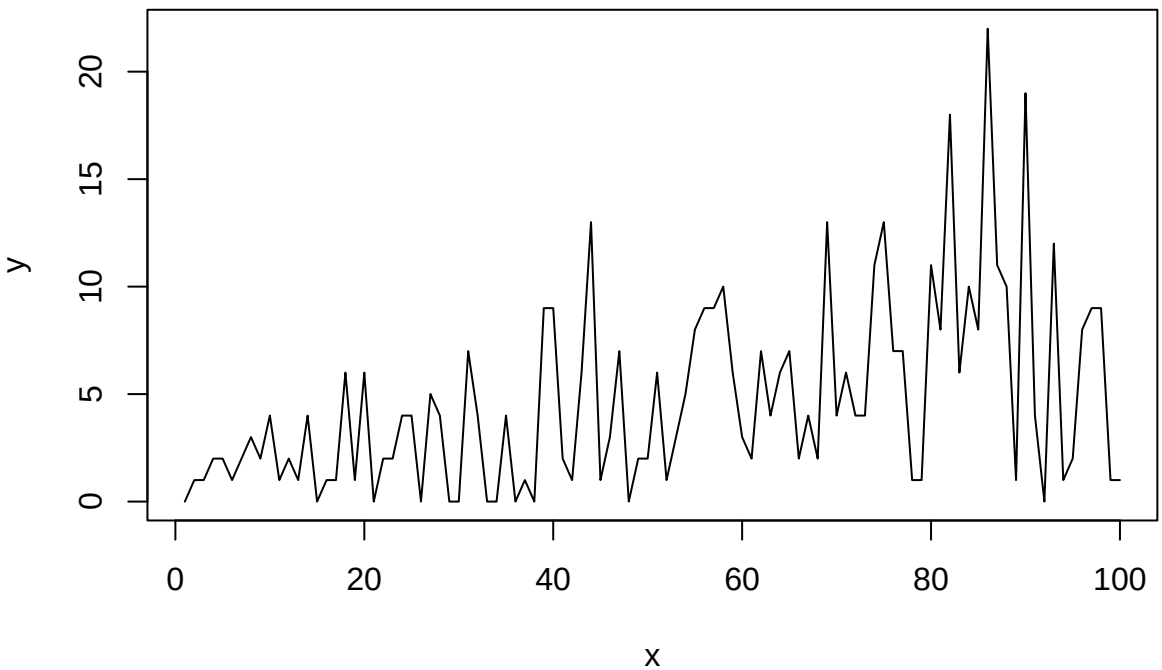
```
## $maximum
## [1] 1.570733
##
```

```
## $objective
## [1] 1
```

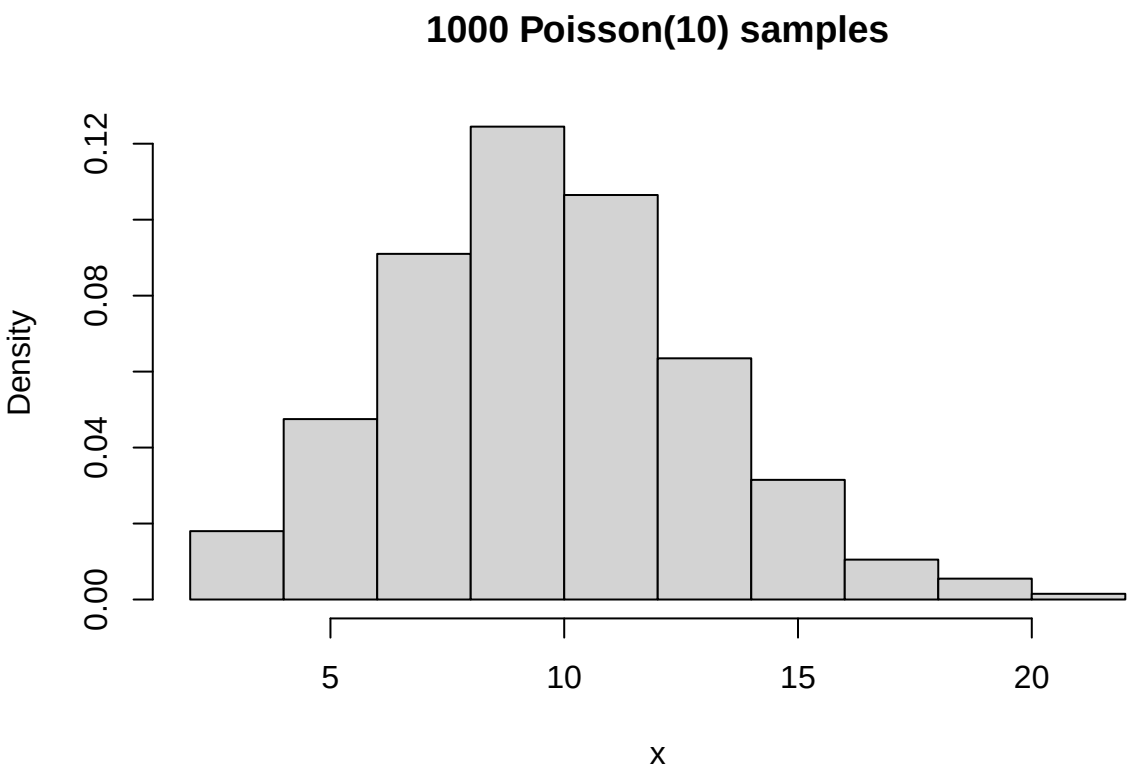
```r
#Writing functions
hit=function(p){
    c=0
    s=0
    while(TRUE){
        x=sample(c(-1,1),1,replace=TRUE,prob=c(1-p,p))
        s=s+x
        c=c+1
        if(s==0){
            break
        }
    }
    c
}
hit(0.5)
```

```
## [1] 10
```

```r
hit_count=function(n){
    f=0
    s=0
    c=0
    while(f<n){
        x=sample(c(-1,1),1,replace=TRUE)
        s=s+x
        if(s==0){
            c=c+1
        }
        f=f+1
    }
    c
}
x=1:100
y=sapply(x,hit_count)
plot(x,y,type="l")
```

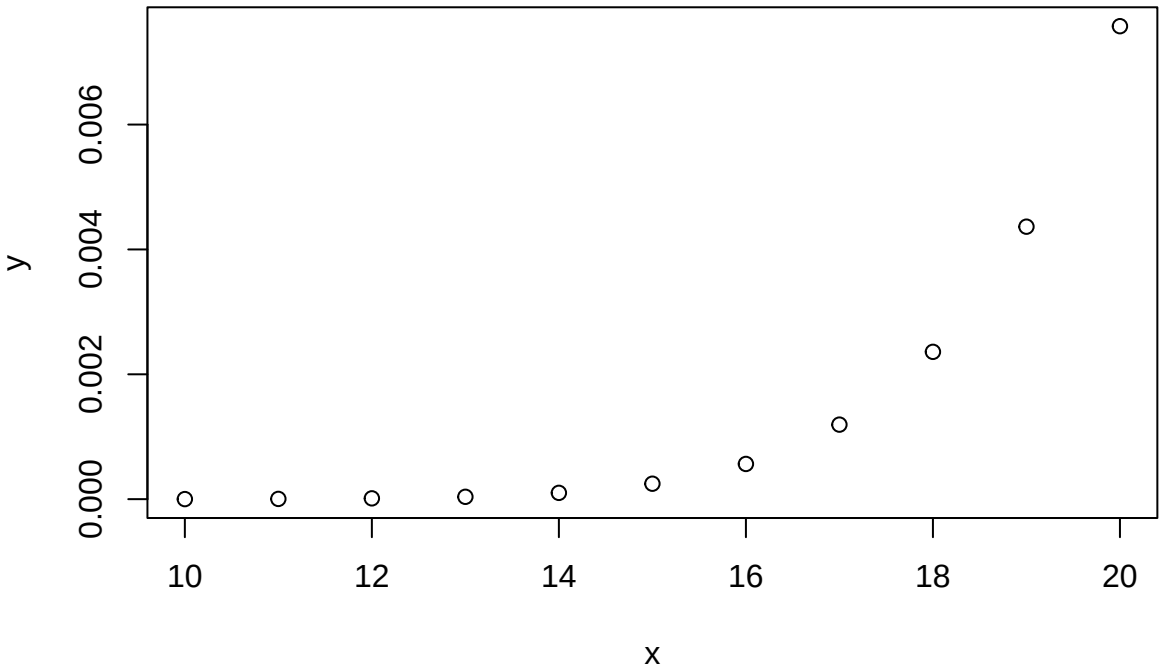



```
#The standard naming scheme for randomness is r* for random number generation,
#d* for pmf/pdf, p* for cdf and q* for quantile
x=rpois(1000,10)
hist(x,probability=TRUE,main="1000 Poisson(10) samples")
```

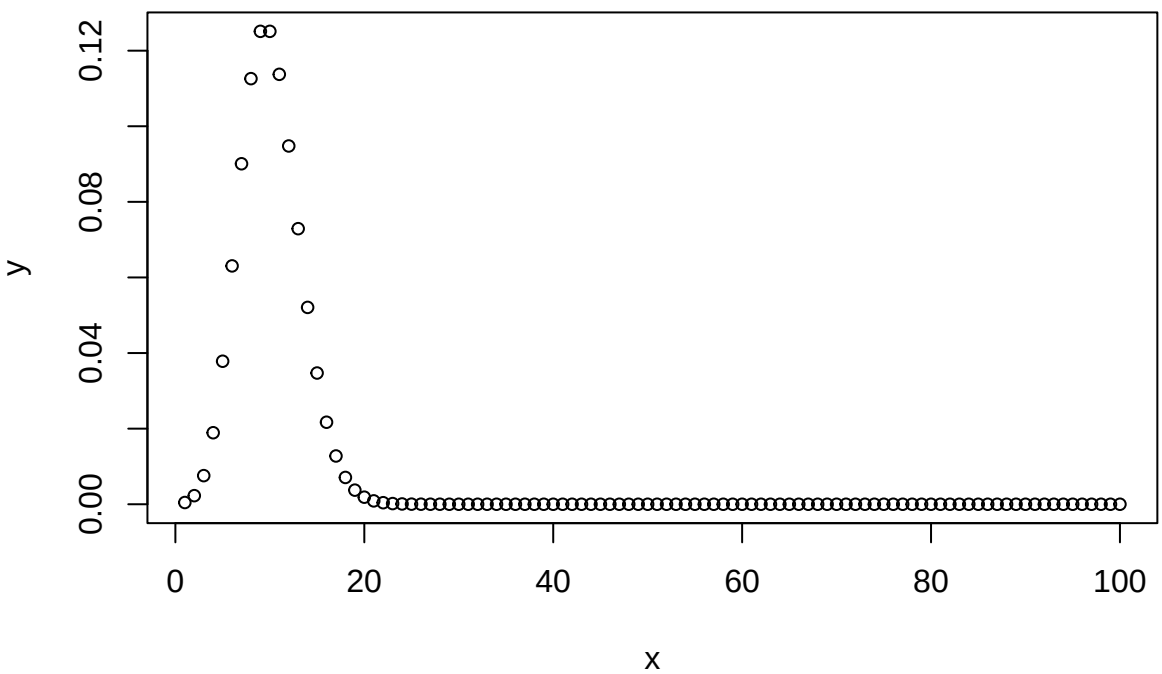



```
#This does not exactly show the probability but weighs the height by the bin
#width so that the sum over the product of height and width over all bins gives 1
x=seq(10,20,by=1)
y=dbinom(x,100,0.3)
```

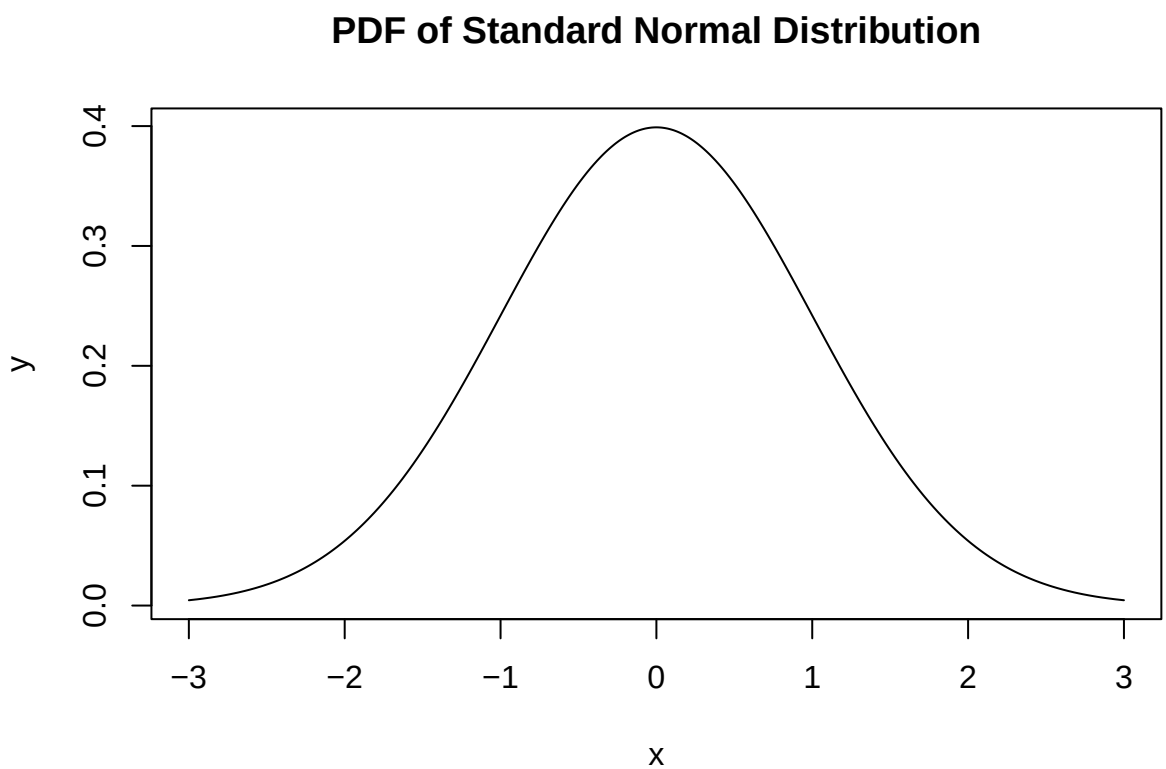
```r
plot(x,y)
```



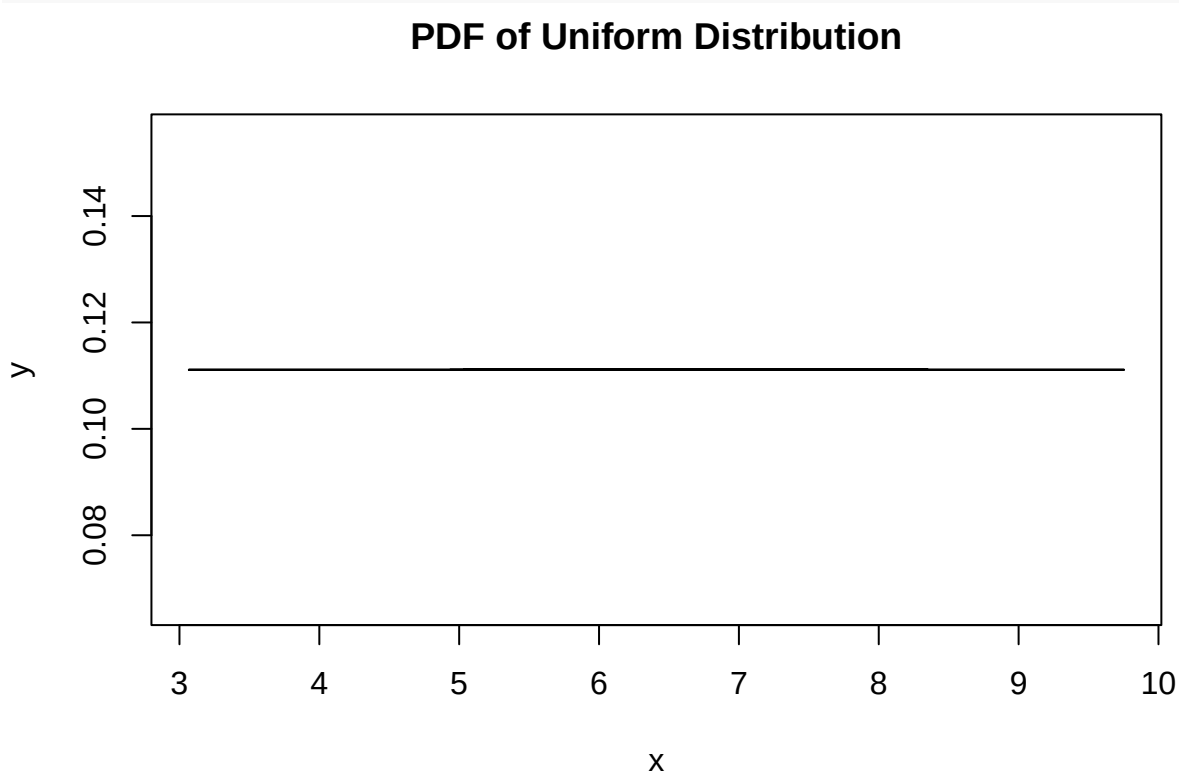

```r
#To control the size of points in plot function use cex, set it to 1 for
#default size, greater than 1 for bigger and less than 1 for smaller
x=1:100
y=dpois(x,10)
plot(x,y,cex=0.8)
```



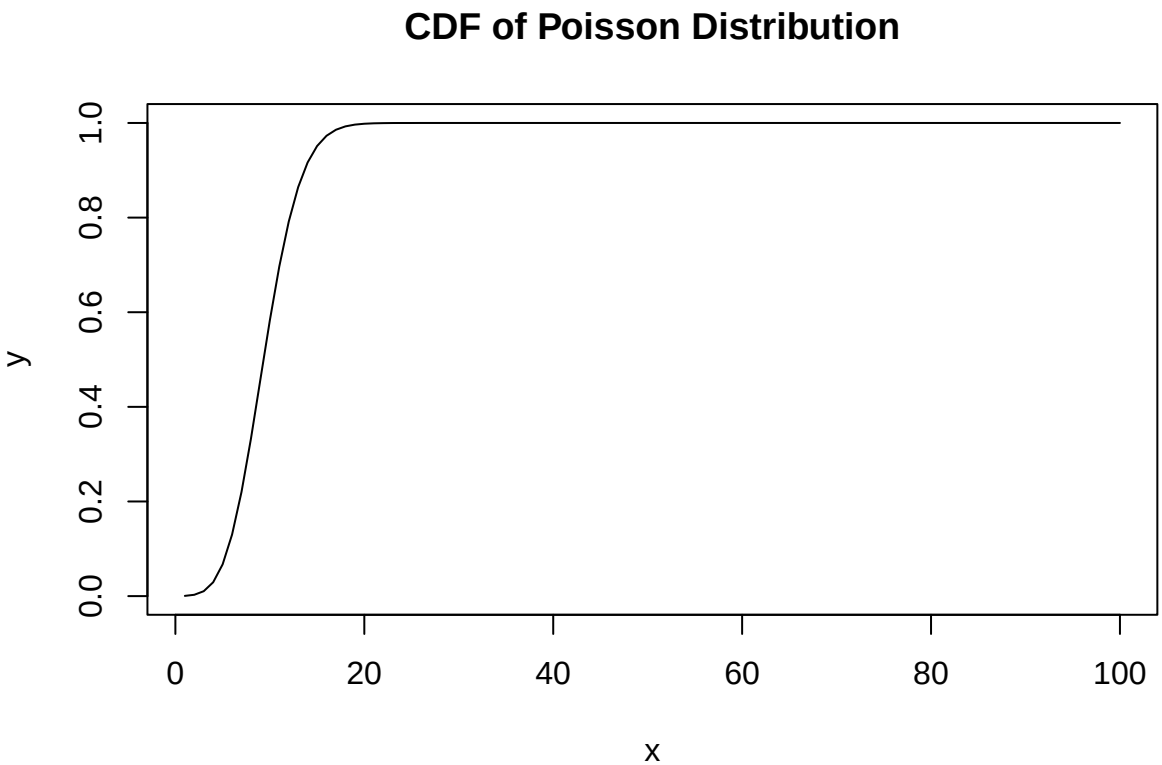

```r
x=seq(-3,3,by=0.01)
y=dnorm(x,0,1)
plot(x,y,type="l",main="PDF of Standard Normal Distribution")
```



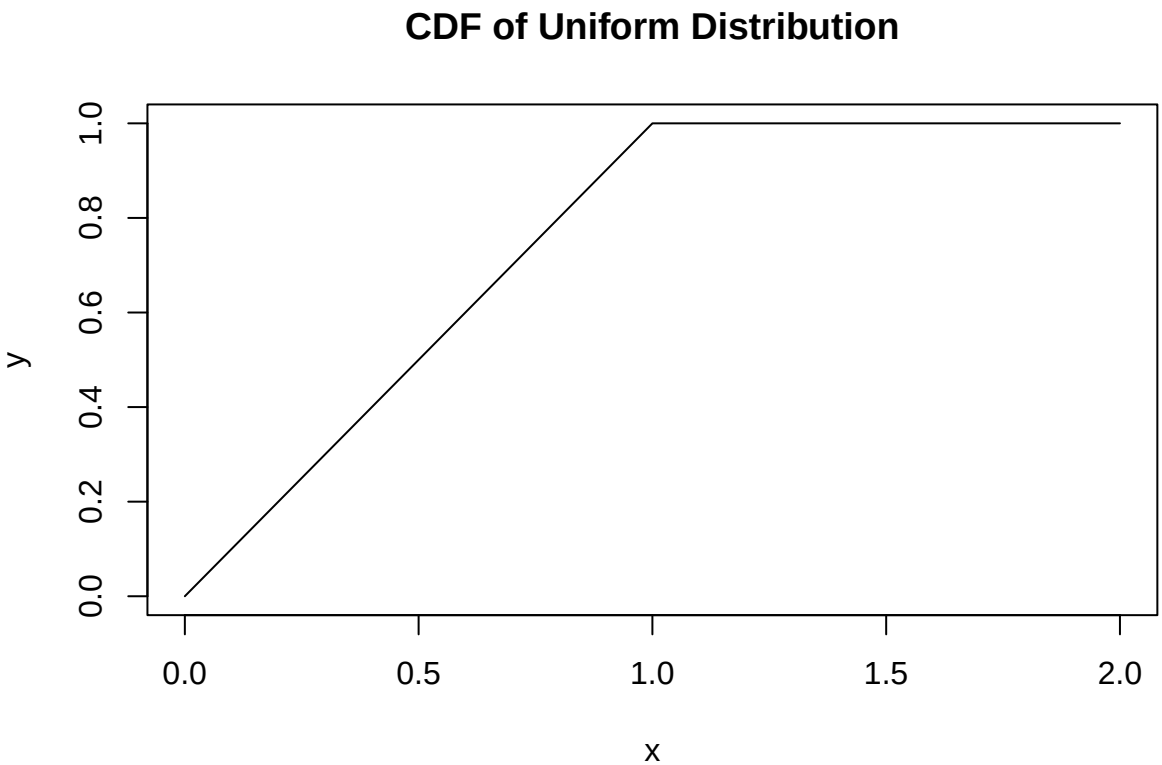

```r
#Simulating uniform distrubution
x=runif(10,min=1,max=10)
y=dunif(x,min=1,max=10)
plot(x,y,type="l",main="PDF of Uniform Distribution")
```



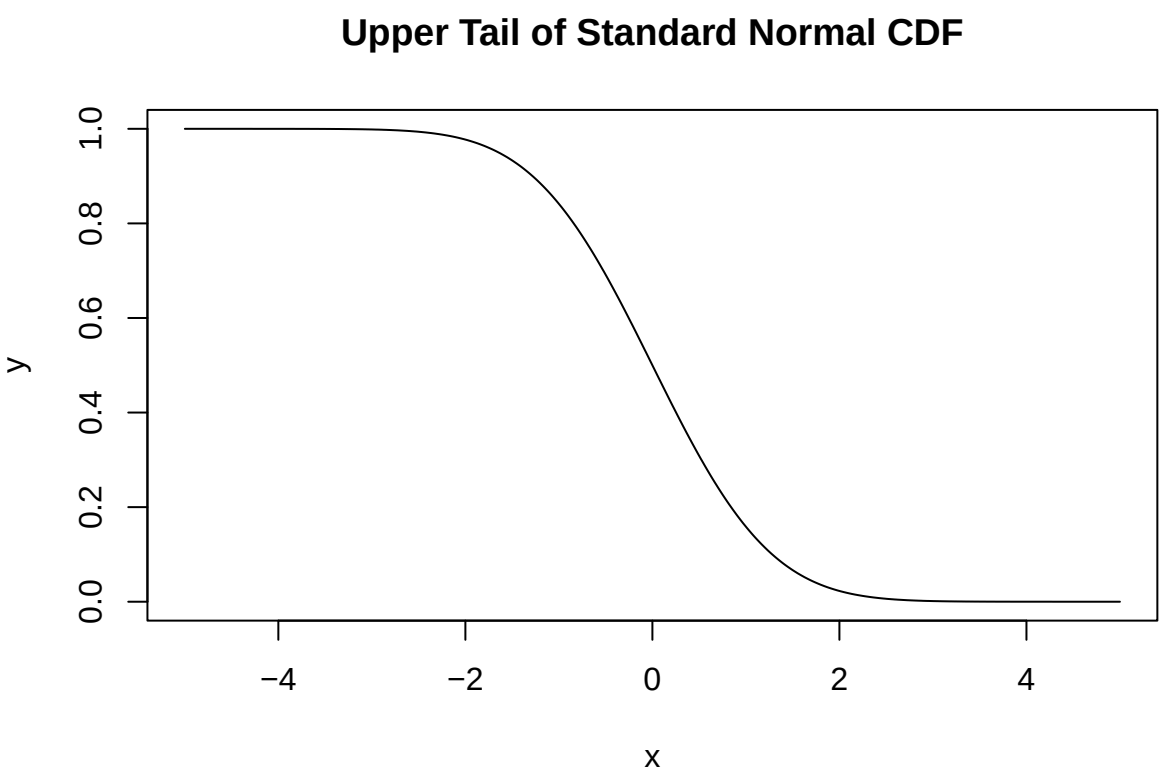
```r
#Plotting the CDF of various distributions
x=1:100
y=ppois(x,10)
plot(x,y,main="CDF of Poisson Distribution",type="l")
```




```r
x=seq(0,2,by=0.001)
y=punif(x)
plot(x,y,type="l",main="CDF of Uniform Distribution")
```




```r
#If we want the upper tail ie. P(X>x) of the CDF we do the following
x=seq(-5,5,by=0.01)
y=pnorm(x,lower.tail=FALSE)
plot(x,y,type="l",main="Upper Tail of Standard Normal CDF")
```



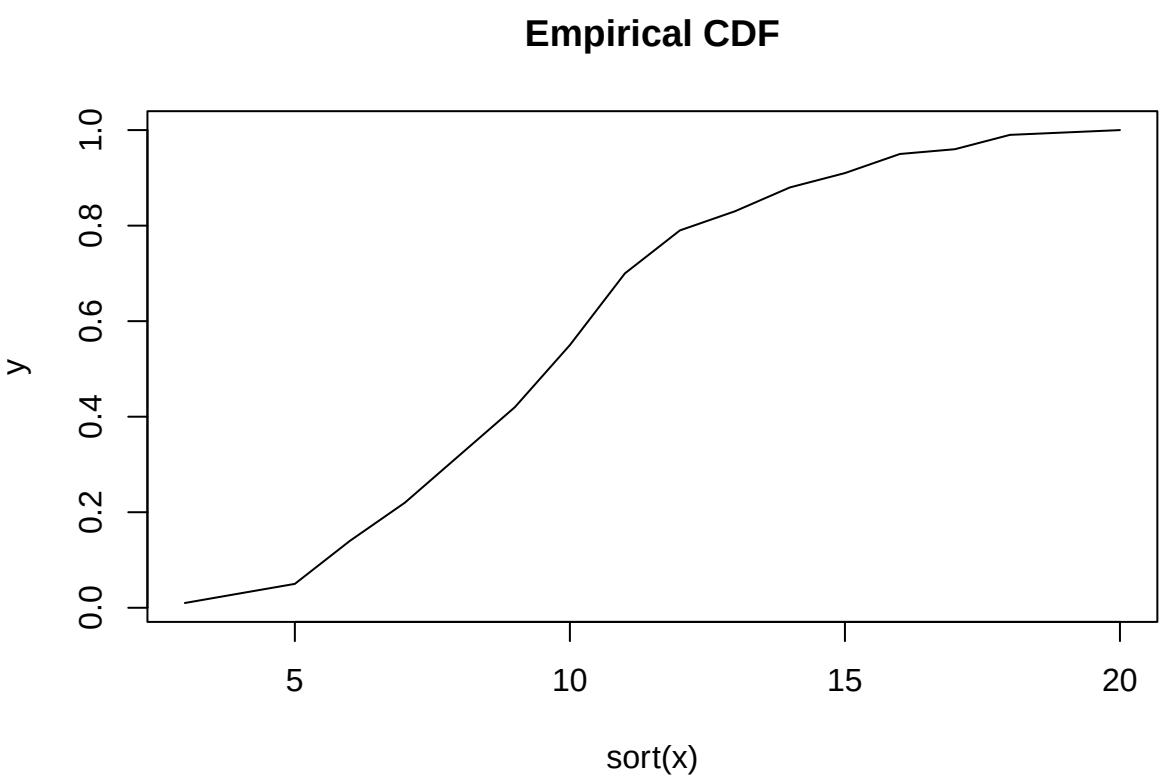
```r
#Generating Frequency table from a given vector
x=c(1,2,2,3,3,3,4,4,4,4,5,5,5,5,5)
table(x)
```

```
## x
## 1 2 3 4 5
## 1 2 3 4 5
```

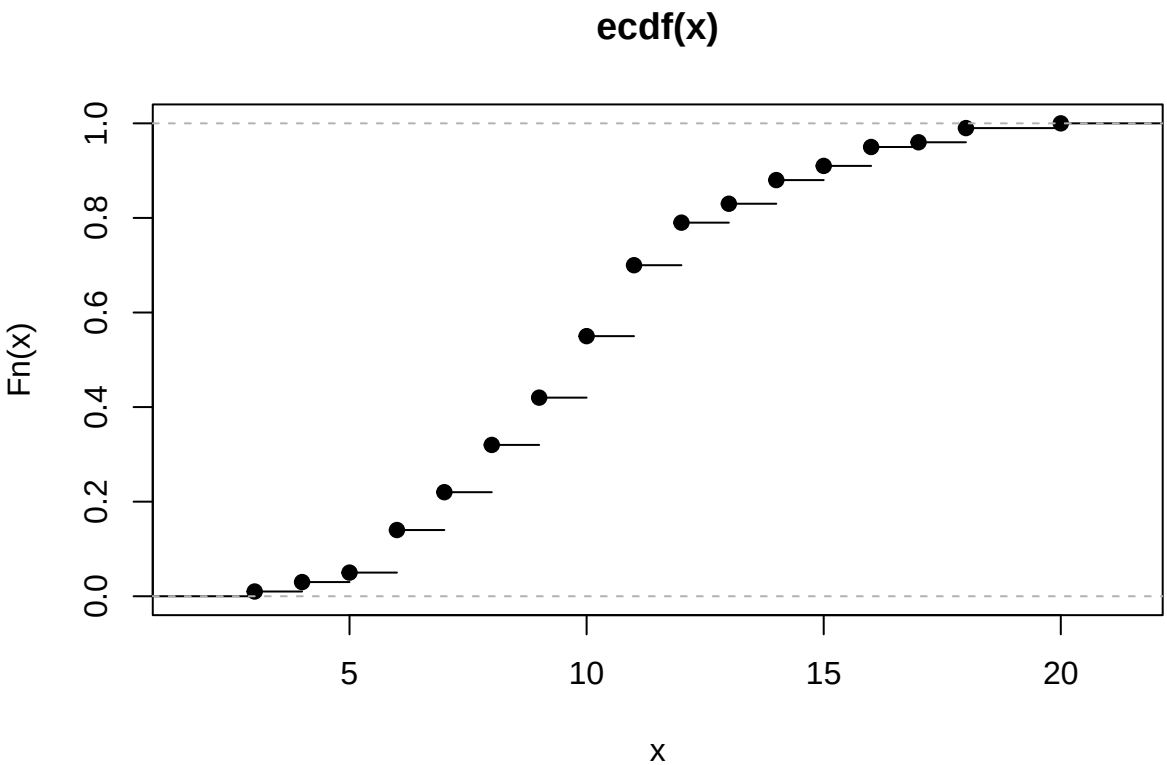
```r
#For generating a table of proportions of frequency
prop.table(table(x))
```

```
## x
##          1          2          3          4          5
## 0.06666667 0.13333333 0.20000000 0.26666667 0.33333333
```

```r
#Empirical CDF
x=rpois(100,10)
f=ecdf(x)
#This is because ecdf(x) returns a function
y=f(sort(x))
plot(sort(x),y,type="l",main="Empirical CDF")
```




```r
#Or we could just plot the ecdf function directly
plot(f)
```




```r
#Density of a distrbution
dnorm(0,0,1)
```

```
## [1] 0.3989423
```

```r
dpois(12,5)
```

```
## [1] 0.00343424
```

```r
dbinom(12,20,0.3)
```

```
## [1] 0.003859282
```

```r
#For evaluating CDF
pnorm(3)-pnorm(-3)
```

```
## [1] 0.9973002
```

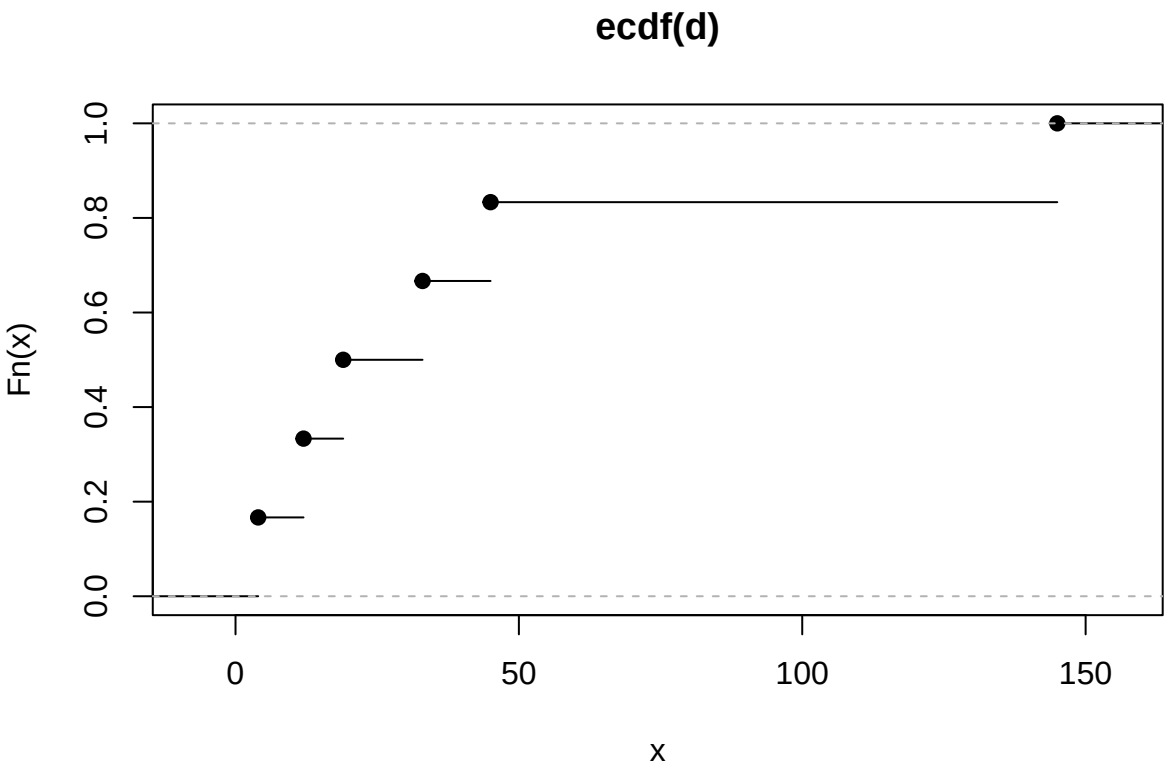
```r
ppois(12,5)
```

```
## [1] 0.9979811
```

```r
#Inverse of CDF
qnorm(0.99,0,1)
```

```
## [1] 2.326348
```

```r
#Empirical CDF plotting, the ECDF function does not return a particular value, instead it returns a fun
d=c(4,45,33,12,145,19)
plot(ecdf(d))
```




```r
#Confidence Interval; suppose we want a 70% confidence interval
qnorm(0.025)
```

```
## [1] -1.959964
```

```r
x=c(1,2,3,4,5,6)
x>4
```

```
## [1] FALSE FALSE FALSE FALSE  TRUE  TRUE
```

```r
x[1]
```

```
## [1] 1
```

```r
summary(x)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    2.25    3.50    3.50    4.75    6.00
```